

Zadanie 2 – Umieźdzynarodowienie aplikacji SwiftUI

Należy utworzyć widok spełniający poniższe wymagania:

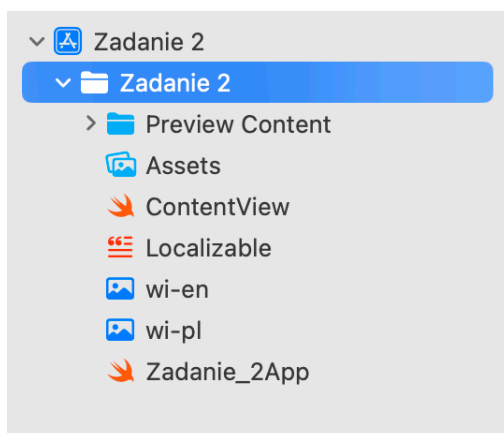
1. Widok wyświetla logo uczelni.
2. Po naciśnięciu przycisku z napisem "Button" wyświetlany jest alert o treści: "The faculty is running %i programs on graduate and undergraduate level.". Alert powinien mieć zmienną liczbę kierunków (zrobioną poprzez %i), domyślnie 4.
3. Po zmianie języka systemu na polski:
 - logo uczelni zostaje podmienione na wersję z polskim napisem,
 - przycisk zmienia tekst na "Przycisk",
 - tekst alertu wyświetlany jest po polsku.



Przykładowy wygląd aplikacji w języku angielskim i polskim.

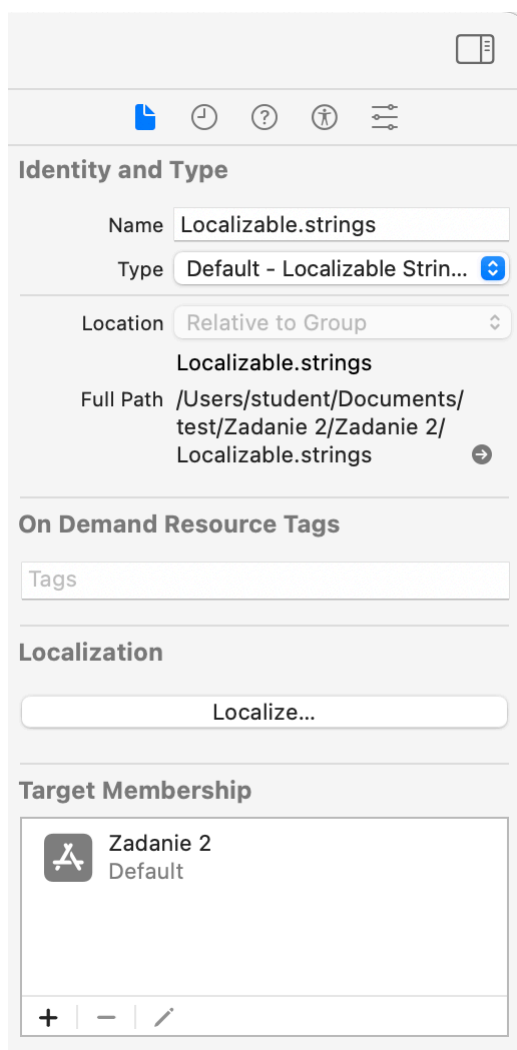
Instrukcja do zadania 2

1. Utwórz nowy projekt SwiftUI. Dodaj plik `Localizable.strings` oraz wgraj obrazy do folderu `Assets.xcassets`. Pamiętaj, że obrazy muszą znajdować się w katalogu `Assets`, aby mogły być poprawnie użyte w kodzie.

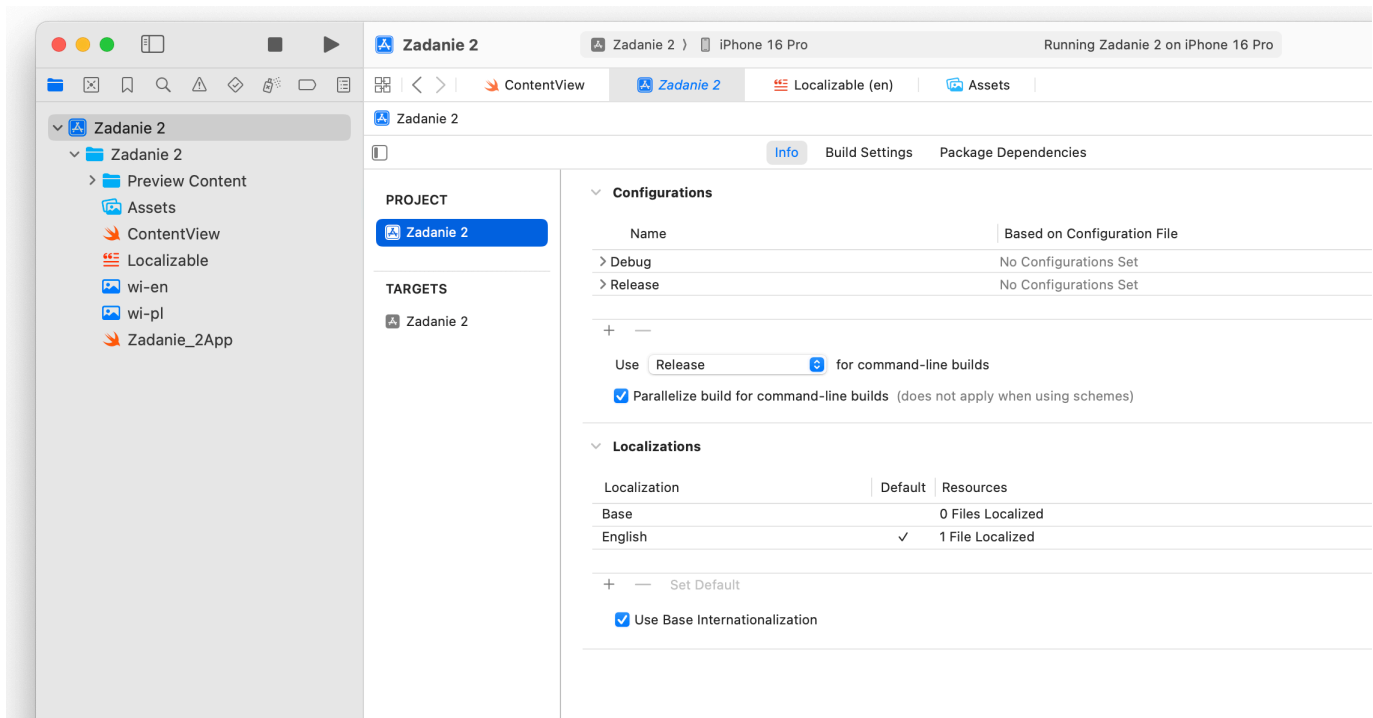


Obrazy wi-en i wi-pl trzeba dodać do katalogu Assets

2. Kliknij w plik `Localizable.strings`, następnie w panelu właściwości wybierz przycisk `Localize...`

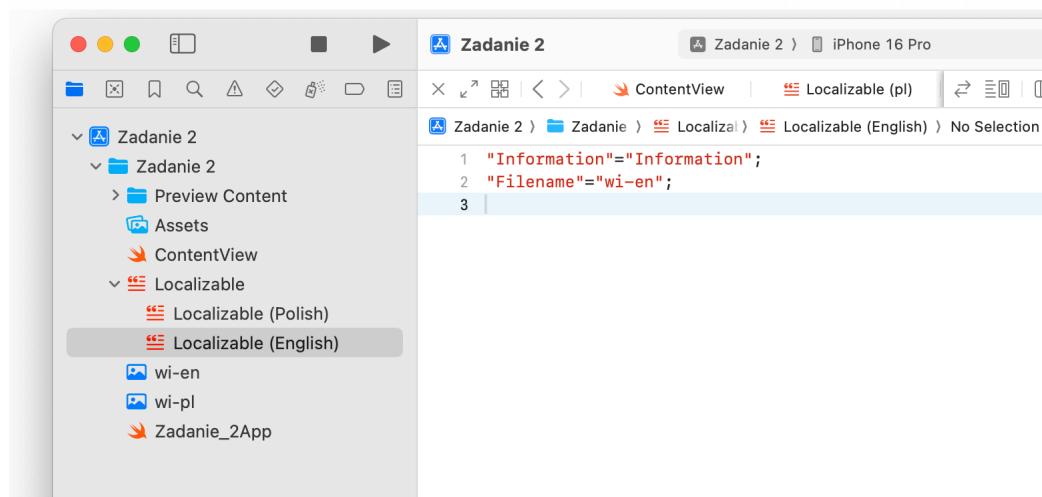


3. W ustawieniach projektu (Project → Info → Localizations) kliknij ikonę + i dodaj język Polish.



Dodawanie nowego języka.

4. Uzupełnij pliki tłumaczeń. W folderze projektu powinny pojawić się dwa pliki: `Localizable.strings` (English) i `Localizable.strings` (Polish).



Przykładowe pliki lokalizacyjne.

5. Testujemy czy lokalizacje działają poprawnie. W tym celu do `ContentView` dodajemy makro `NSLocalizedString`. Pierwszy parametr tego makra to klucz z pliku `Localizable.strings`, drugim parametrem jest komentarz, który powinien wyjaśniać czym jest dana tłumaczona wartość. Wyciągamy zlokalizowany tekst przycisku i dodajemy do widoku. Można też użyć skróconego inicjalizatora `Text("klucz")`. Minimalny, działający przykład kodu będzie wyglądać w ten sposób:

```
1 var body: some View {
2     VStack {
3         Text(NSLocalizedString("Button", comment: "Button label"))
4     }
5 }
```

6. Jeżeli tekst przycisku zmienia się w zależności od języka, to przejdźmy do przetestowania obrazów. Dodajmy do `VStack` kontrolkę `Image` i podajmy jej wartość `"wi-en"`. Jeżeli obrazy zostały poprawnie dodane do assetów to powinien wyświetlić się obraz.

7. Jeżeli poprawnie wyświetla się obraz w aplikacji, to użyj makra `NSLocalizedString`, aby pobierać odpowiedni plik w zależności od języka. Można je podać bezpośrednio lub przypisać do zmiennej i ją przekazać do konstrukcji `Image`.

```
1 NSLocalizedString("Filename", comment: "Localized filename")
```

8. Dodaj przycisk i alert. Dodaj prywatną zmienną typu `bool` z atrybutem `State` do kontrolowania widoczności alertu. Składnia dodawania alertów wygląda następująco:

```
1 .alert(tytułAlertu, isPresented: $zmiennaTypuBool) {  
2     Button("OK", role: .cancel) {}  
3 }
```

9. Dodaj stałą typu `int` odpowiedzialną za przechowywanie liczby dostępnych kierunków na uczelni. Dołącz formatowanie do tekstu alertu by uwzględnić tą stałą.

10. Dokończ lokalizację aplikacji, uruchom projekt i zmień język symulatora. Sprawdź, czy tekst i obrazy zmieniają się zgodnie z językiem systemu.